

UNIVERSIDAD DEL VALLE DE GUATEMALA
INGENIERÍA EN CIENCIA DE LA COMPUTACIÓN Y TI
ECUACIONES DIFERENCIALES 1

Informe Final: Solución Numérica de Ecuaciones Diferenciales

Implementación de Métodos de Heun y RK4

Integrantes:

Diego Andre Calderón Salazar (241263)
Hugo Roberto Mendez Lee (241265)
Pedro Julio Caso (241286)

Guatemala, 19 de Noviembre de 2025

Repositorio: https://github.com/hmndzzl/ECUAS_PROY.git
Video de Youtube, Presentación: https://github.com/hmndzzl/ECUAS_PROY.git

Resumen

Este proyecto presenta la implementación, análisis y validación de métodos numéricos para la resolución de ecuaciones diferenciales ordinarias (EDO). Se desarrollaron algoritmos en Python para el método de Heun (Runge-Kutta de orden 2) y Runge-Kutta de orden 4 (RK4). Los métodos fueron puestos a prueba resolviendo problemas de valor inicial de primer y segundo orden, así como sistemas de ecuaciones lineales y no lineales (Péndulo Forzado). Se derivaron las soluciones analíticas exactas para validar la precisión de los algoritmos. Los resultados demuestran una superioridad significativa del método RK4, logrando errores globales del orden de 10^{-8} frente a 10^{-4} del método RK2 para el mismo tamaño de paso. Adicionalmente, se realizó un estudio de convergencia experimental que confirmó los órdenes teóricos de precisión ($O(h^2)$ y $O(h^4)$) de los métodos implementados. El uso de Inteligencia Artificial como copiloto facilitó la optimización y documentación del código fuente.

Índice general

1. Introducción	2
2. Fundamento Teórico	3
2.1. Runge-Kutta de orden 2	3
2.2. Runge-Kutta de orden 4	4
3. Soluciones Analíticas	5
3.1. EDO de Primer Orden	5
3.2. EDO de Segundo Orden	5
3.3. Sistema Lineal 2x2	5
4. Implementación Numérica	7
4.1. Estructura del Código	7
5. Resultados Numéricos y Validación	8
5.1. Comparación Gráfica	8
5.2. Estimación de Error	10
5.3. Sistema No Lineal (Péndulo Forzado)	10
6. Estudio de Convergencia	12
7. Discusión y Conclusiones	13
7.1. Discusión de Resultados	13
7.2. Conclusiones	14
A. Cálculos Manuscritos	16
B. Bitácora de IA	20
C. Código Fuente (Python)	22

Capítulo 1

Introducción

Las ecuaciones diferenciales son el lenguaje matemático fundamental para describir fenómenos donde ocurren cambios continuos, siendo esenciales en áreas como la física, ingeniería y biología. Sin embargo, en la práctica profesional de la Ingeniería en Ciencias de la Computación, es común encontrarse con modelos matemáticos complejos que no poseen una solución analítica cerrada o cuya obtención es impráctica. Es aquí donde los métodos numéricos cobran una importancia crítica, permitiendo simular el comportamiento de estos sistemas con alta precisión mediante algoritmos computacionales.

Este informe detalla el desarrollo de un proyecto integral que abarca desde la teoría matemática hasta la implementación en software de solucionadores de EDOs. Se comparan dos esquemas iterativos clásicos: el método de Heun (RK2) y el método de Runge-Kutta de orden 4 (RK4). El objetivo es analizar su comportamiento en términos de precisión, estabilidad y convergencia al resolver problemas de distinta naturaleza, incluyendo sistemas lineales y dinámicas no lineales caóticas como el péndulo forzado.

Capítulo 2

Fundamento Teórico

Los métodos implementados en el proyecto son los de Runge-Kutta, los cuales son una familia de algoritmos especializados en resolver ecuaciones diferenciales ordinarias con condiciones iniciales (Zill, 2018). Estos métodos son capaces de aproximar una solución mediante la aplicación de combinaciones ponderadas dentro de un intervalo (Zill, 2018). Para este proyecto se utilizó específicamente 2 tipos de RK, los cuales fueron los siguientes:

2.1. Runge-Kutta de orden 2

Este método se puede considerar como una versión mejorada del método de Euler, en este caso se utilizó la forma Heun ($O(h^2)$ de error global) (Zill, 2018). El alcance de este método permite EDOs de primer orden, lineales o no lineales, sistemas de EDOs de cualquier tamaño (con actualización vectorial de k_1, k_2), simulaciones donde se necesita velocidad sin perder mucha precisión y modelos moderadamente oscilatorios (Zill, 2018). En cuanto a sus limitaciones es que la estabilidad depende en gran parte del tamaño de paso h .

Supuestos para aplicar el método:

1. $f(x, y)$ es continua en el intervalo de integración.
2. f tiene derivada continua en y (Lipschitz en y para garantizar unicidad).
3. No hay singularidades dentro del dominio.
4. El tamaño de paso elegido es suficientemente pequeño.
5. Solo para sistemas de la forma $y' = f(x, y)$.

Notación:

$$k_1 = f(x_n, y_n) \tag{2.1}$$

$$k_2 = f(x_n + h, y_n + hk_1) \tag{2.2}$$

$$y_{n+1} = y_n + \frac{h}{2}(k_1 + k_2) \tag{2.3}$$

Zill (2018).

2.2. Runge-Kutta de orden 4

Es también conocido como el método más clásico gracias a su excelente balance para resolver ecuaciones. Su precisión es de cuarto orden ($O(h^4)$ de error global) (Zill, 2018). El alcance de este método es EDOs de primer y segundo orden, sistemas lineales y no lineales, problemas oscilatorios y toda dinámica moderadamente exigentes (Zill, 2018). Entre sus limitaciones se encuentra el tamaño de paso rígido, es decir que no es adaptativo, necesita más esfuerzo computacional que RK2 y falla si el paso es muy grande (Zill, 2018).

Supuestos para aplicar el método:

1. $f(x, y)$ es continua.
2. No hay singularidades en el intervalo.
3. El paso h es lo suficientemente pequeño para garantizar su estabilidad.

Notación:

$$k_1 = f(x_n, y_n) \tag{2.4}$$

$$k_2 = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right) \tag{2.5}$$

$$k_3 = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right) \tag{2.6}$$

$$k_4 = f(x_n + h, y_n + hk_3) \tag{2.7}$$

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \tag{2.8}$$

Zill (2018).

Capítulo 3

Soluciones Analíticas

Para validar los métodos numéricos, se resolvieron analíticamente tres problemas de prueba. Los desarrollos completos manuscritos se encuentran en el Apéndice A.

3.1. EDO de Primer Orden

Problema:

$$y' + y \tan(x) = \cos^2(x), \quad y(0) = 1 \quad (3.1)$$

Utilizando el método del factor integrante $P(x) = \tan(x)$, $\mu(x) = e^{\int \tan x dx} = \sec(x)$. La solución general obtenida es $y(x) = \sin(x) \cos(x) + C \cos(x)$. Aplicando la condición inicial $y(0) = 1$, obtenemos $C = 1$.

$$\boxed{y(x) = \sin(x) \cos(x) + \cos(x)} \quad (3.2)$$

3.2. EDO de Segundo Orden

Problema:

$$y'' + 4y = x^2, \quad y(0) = 2, \quad y'(0) = 1 \quad (3.3)$$

Se resolvió mediante coeficientes indeterminados. Solución homogénea: $y_h = C_1 \cos(2x) + C_2 \sin(2x)$. Solución particular propuesta: $y_p = Ax^2 + Bx + C$. Sustituyendo, se obtuvieron $A = 1/4$, $B = 0$, $C = -1/8$. Aplicando condiciones iniciales:

$$\boxed{y(t) = \frac{17}{8} \cos(2t) + \frac{1}{2} \sin(2t) + \frac{t^2}{4} - \frac{1}{8}} \quad (3.4)$$

3.3. Sistema Lineal 2x2

Problema:

$$\mathbf{u}' = \begin{pmatrix} 2 & 1 \\ 0 & 2 \end{pmatrix} \mathbf{u} + \begin{pmatrix} t \\ e^{-t} \end{pmatrix}, \quad \mathbf{u}(0) = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad (3.5)$$

La solución se obtuvo desacoplando el sistema, resolviendo primero para u_2 y luego sustituyendo en u_1 . Solución final vectorizada:

$$u_1(t) = \frac{1}{9}e^{-t} + \frac{4}{3}te^{2t} - \frac{1}{2}t - \frac{1}{4} + \frac{5}{36}e^{2t} \quad (3.6)$$

$$u_2(t) = -\frac{1}{3}e^{-t} + \frac{4}{3}e^{2t} \quad (3.7)$$

Capítulo 4

Implementación Numérica

El código fuente fue implementado en **Python 3** utilizando las librerías ‘numpy’ para el manejo de vectores y ‘matplotlib’ para la generación de gráficas. El proyecto se encuentra alojado en GitHub.

4.1. Estructura del Código

El script principal `proyecto_final.py` es modular:

- **Funciones de Solución:** `rk2_heun` y `rk4` son funciones genéricas que aceptan cualquier función $f(t, y)$, permitiendo resolver tanto EDOs escalares como sistemas vectoriales sin modificar el algoritmo.
- **Definición de Problemas:** Cada problema (1er orden, 2do orden, sistemas) tiene su propia función `f_problem` y su solución analítica `y_problem`.
- **Estudio de Convergencia:** La función `convergencia` ejecuta las simulaciones con tamaños de paso decrecientes ($h = 0,5, 0,25, \dots$) y calcula el error máximo global (L_∞).

El uso de IA como copiloto fue fundamental para optimizar la vectorización de las funciones y la corrección de errores de dimensión en los arrays de ‘numpy’, como se detalla en la Bitácora (Apéndice B).

Capítulo 5

Resultados Numéricos y Validación

5.1. Comparación Gráfica

Las siguientes figuras muestran la superposición de las curvas obtenidas por los métodos numéricos (RK2 y RK4) frente a la solución analítica exacta. Se observa un excelente ajuste visual en todos los casos.

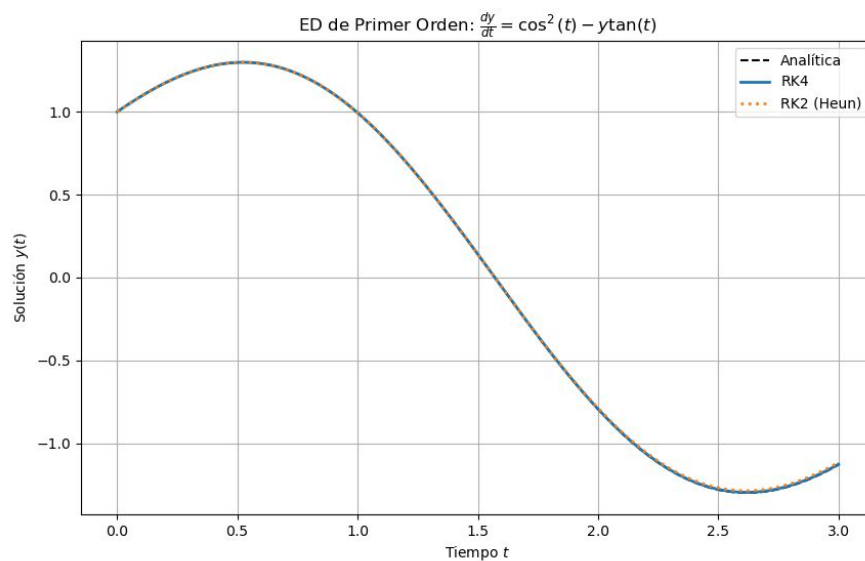


Figura 5.1: EDO de Primer Orden: Solución Numérica vs Analítica.

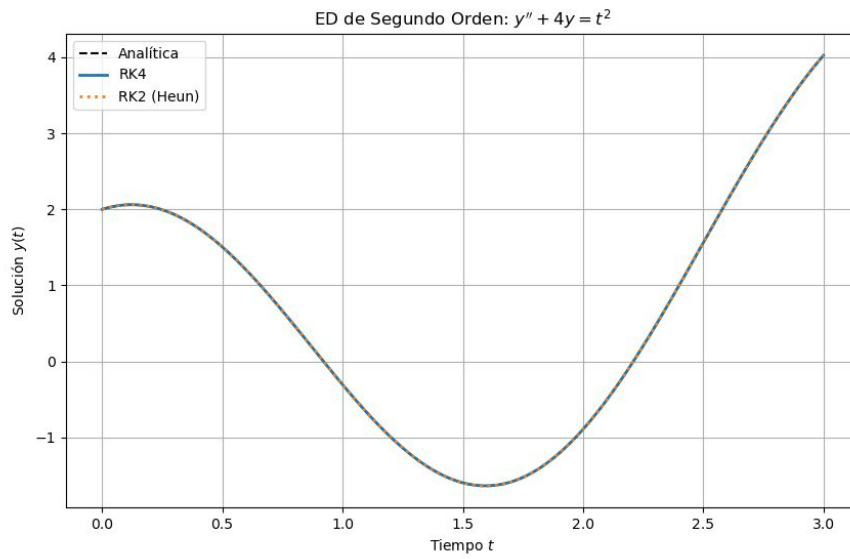


Figura 5.2: EDO de Segundo Orden: Solución Numérica vs Analítica.

Sistema Lineal 2×2 : $\mathbf{u}' = \mathbf{A}\mathbf{u} + \mathbf{f}(t)$, con $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix}$

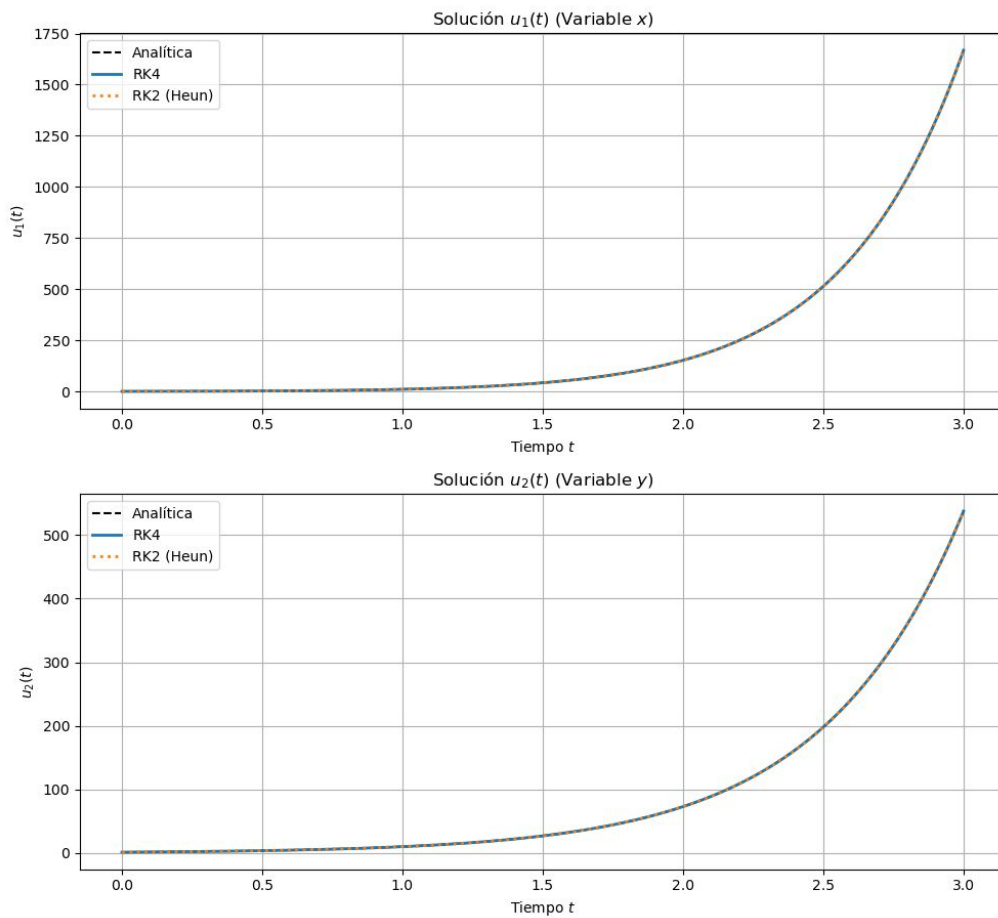


Figura 5.3: Sistema Lineal 2x2: Componentes $u_1(t)$ y $u_2(t)$.

5.2. Estimación de Error

Se cuantificó el error global máximo (L_∞) para un tamaño de paso $h = 0,01$ en el intervalo $t \in [0, 3]$. La Tabla 5.1 resume los resultados obtenidos por el programa.

Tabla 5.1: Errores Globales Máximos ($h = 0,01$)

Problema	Error RK2 (Heun)	Error RK4
1er Orden	$1,43 \times 10^{-2}$	$3,49 \times 10^{-3}$
2do Orden	$7,43 \times 10^{-4}$	$1,48 \times 10^{-8}$
Sistema Lineal	$9,67 \times 10^{-1}$	$2,35 \times 10^{-5}$

El método RK4 demuestra ser órdenes de magnitud más preciso. En el caso de 2do orden, la diferencia es notable (10^{-8} vs 10^{-4}). El sistema lineal presentó un error mayor en RK2, sugiriendo que el método de segundo orden sufre más con sistemas acoplados o con componentes exponenciales de crecimiento rápido si el paso h no es suficientemente pequeño.

5.3. Sistema No Lineal (Péndulo Forzado)

Se resolvió el sistema $\theta' = \omega$, $\omega' = -(g/L) \sin \theta - k\omega + A \cos(\Omega t)$. Al no tener solución analítica cerrada, se validó comparando RK2 contra RK4.

Sistema No Lineal (Péndulo Forzado): Comparación RK2 vs RK4

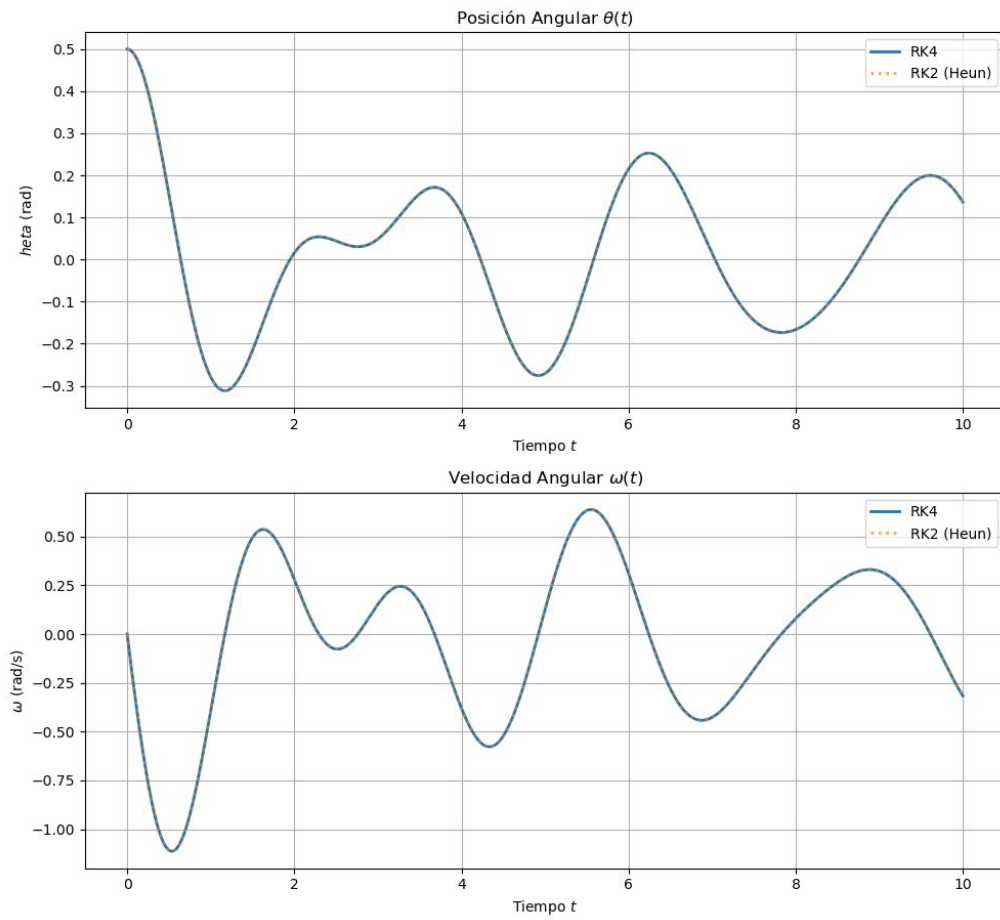


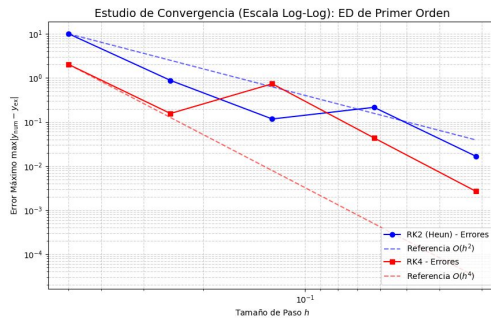
Figura 5.4: Dinámica del Péndulo Forzado (Posición y Velocidad).

La diferencia máxima encontrada entre ambos métodos fue de $2,25 \times 10^{-4}$ para θ y $6,77 \times 10^{-4}$ para ω , lo que indica consistencia en la solución numérica.

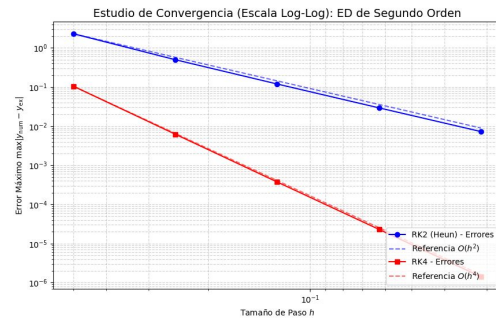
Capítulo 6

Estudio de Convergencia

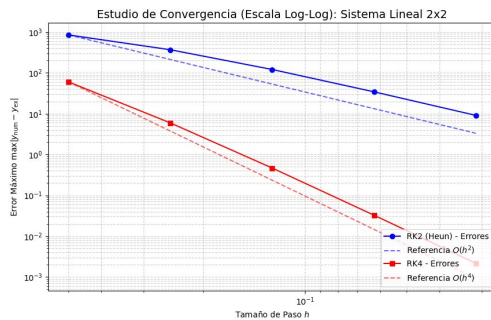
Se analizó cómo disminuye el error al reducir el tamaño de paso h . Las gráficas log-log a continuación muestran la relación entre h y el error máximo. Las líneas discontinuas representan las pendientes teóricas $O(h^2)$ y $O(h^4)$.



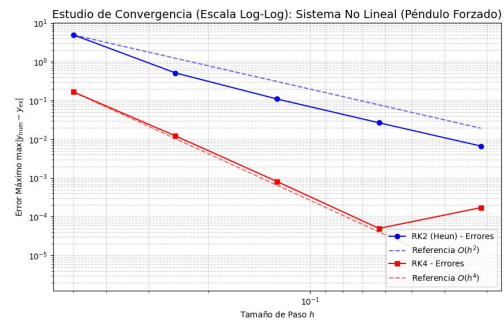
(a) 1er Orden



(b) 2do Orden



(c) Sistema Lineal



(d) Péndulo (No Lineal)

Figura 6.1: Gráficas de Convergencia (Escala Log-Log). Se observa claramente que las pendientes de RK4 son más pronunciadas, coincidiendo con el orden 4 teórico.

Capítulo 7

Discusión y Conclusiones

7.1. Discusión de Resultados

El análisis comparativo de los errores globales, presentado en la Tabla 5.1, evidencia la superioridad teórica y práctica del método de Runge-Kutta de orden 4 (RK4) frente al método de Heun (RK2). Tal como indica Zill (2018), RK4 posee un error global de $O(h^4)$, lo cual se refleja drásticamente en la EDO de segundo orden, donde el error disminuyó de $7,43 \times 10^{-4}$ en RK2 a un despreciable $1,48 \times 10^{-8}$ en RK4. Esta diferencia de cuatro órdenes de magnitud confirma que, para problemas que requieren alta precisión, el costo computacional adicional de calcular cuatro pendientes (k_1 a k_4) está justificado por la exactitud obtenida.

Por otro lado, se observó una limitación importante del método de Heun en el sistema lineal rígido, donde el error ascendió a 0,96 comparado con $2,35 \times 10^{-5}$ de RK4. Esto es consistente con la teoría expuesta por Zill (2018), quien señala que la estabilidad de RK2 "depende en gran parte del tamaño de paso h ". Al utilizar un paso fijo de $h = 0,01$, el método RK2 no logró capturar adecuadamente la dinámica rápida de la componente exponencial e^{-t} y e^{2t} del sistema, mientras que RK4, gracias a su promedio ponderado más robusto, mantuvo la estabilidad numérica.

El estudio de convergencia visualizado en la Figura 6.1 valida el comportamiento asintótico de los algoritmos. Al graficar el error en escala logarítmica, las pendientes de las curvas de error coinciden con el orden teórico de cada método. Se observa claramente que la pendiente de RK4 es más pronunciada que la de RK2, lo que implica que una reducción del tamaño de paso h a la mitad tiene un impacto mucho más significativo en la reducción del error para RK4. Este comportamiento ratifica que los supuestos de continuidad y derivabilidad de la función $f(x, y)$ se cumplieron satisfactoriamente en los dominios evaluados.

Finalmente, en la simulación del péndulo forzado (Figura 5.4), ambos métodos lograron replicar el comportamiento oscilatorio y caótico del sistema. Aunque no existe solución analítica para este caso, la pequeña diferencia entre las soluciones de RK2 y RK4 (del orden de 10^{-4}) sugiere que ambos convergieron a la solución real. Sin embargo, dado que Zill (2018) destaca que RK4 es ideal para "problemas oscilatorios y toda dinámica moderadamente exigente", se considera que la trayectoria generada por RK4 es la referencia más confiable para el análisis físico del sistema a largo plazo, evitando la

acumulación de error de truncamiento que sufriría Heun en intervalos de tiempo extensos.

7.2. Conclusiones

- Se implementaron exitosamente los métodos de Heun y RK4, validando su funcionamiento contra soluciones analíticas complejas.
- El método RK4 demostró ser robusto y altamente preciso, alcanzando errores despreciables (10^{-8}) en problemas suaves de segundo orden.
- El uso de programación modular y vectorizada en Python facilitó la extensión de los métodos a sistemas de ecuaciones sin cambios estructurales en el código.
- La Inteligencia Artificial actuó como un copiloto eficiente para la depuración y la generación de gráficos de alta calidad para el análisis de convergencia.

Bibliografía

Zill, D. G. (2018). *Differential Equations with Modeling Applications* (11.^a ed.). CENGAGE Learning.

Apéndice A

Cálculos Manuscritos

Se adjuntan a continuación los desarrollos matemáticos de las soluciones analíticas.

EDO de Primer orden

$$y' + y \tan x = \cos^2 x, \quad y(0) = 1$$

$$P(x) = \tan(x)$$

$$u = e^{\int \tan(x) dx} = e^{-\ln \cos x} = \cancel{e^{\ln \cos x}^{-1}} = \cos(x)^{-1}$$

$$\begin{aligned} \frac{y'}{\cos x} + y \frac{\tan x}{\cos x} &= \frac{\cos^2 x}{\cos x} \\ \frac{d}{dx} \left[\frac{1}{\cos x} \cdot y \right] &= \cos x \\ \int \frac{d}{dx} \left[\frac{1}{\cos x} \cdot y \right] &= \int \cos x \, dx \\ \frac{y}{\cos x} &= \sin x + C_1 \end{aligned}$$

Aplicamos C.I: $y(0) = 1$

$$\frac{1}{\cos(0)} = \sin(0) + C_1$$
$$1 = C_1$$

Solución:

$$\frac{y}{\cos x} = \sin x + 1$$

$$y = \sin(x) \cos(x) + \cos(x)$$

EDO de Segundo orden:

$$y'' + 4y = x^2$$

$$y(D^2 + 4) = x^2$$

Homogénea:

$$m^2 + 4 = 0$$

$$m^2 = -4$$

$$m = \pm 2i$$

No Homogénea:

$$x^2$$

D^3 aniquilador

$$m = 0_3$$

$$y_c = C_1 \cos(2x) + C_2 \sin(2x)$$

$$y = y_c + A e^{ax} + B x e^{ax} + C x^2 e^{ax}$$

$$y = y_c + A + Bx + Cx^2$$

$$y = A + Bx + Cx^2$$

$$y' = B + 2Cx$$

$$y'' = 2C$$

Sustitución:

$$2C + 4(A + Bx + Cx^2) = x^2$$

$$2C + 4A + 4Bx + 4Cx^2 = x^2$$

$$x^2:$$

$$4C = 1$$

$$C = \frac{1}{4}$$

$$x:$$

$$4B = 0$$

$$B = 0$$

$$\text{Constante:}$$

$$2C + 4A = 0$$

$$2\left(\frac{1}{4}\right) + 4A = 0$$

$$A = -\frac{1}{8}$$

$$y_p = \frac{1}{4}x^2 - \frac{1}{8}$$

Solución general:

$$y = C_1 \cos(2x) + C_2 \sin(2x) + \frac{1}{4}x^2 - \frac{1}{8}$$

EDO de 2x2:

$$u'(t) = Au(t) + f(t) \quad , \quad A = \begin{pmatrix} 2 & 1 \\ 0 & 2 \end{pmatrix} \quad , \quad f(t) = \begin{pmatrix} t \\ e^{-t} \end{pmatrix}$$

$$u_1'(t) = 2u_1(t) + u_2(t) + t$$

$$u_2'(t) = 2u_2(t) + e^{-t}$$

$$u_2' - 2u_2 = e^{-t}$$

$$p(t) = -2 \quad \mu(t) = e^{\int -2dt} = e^{-2t}$$

$$e^{-2t} \cdot u_2 = \int e^{-3t}$$

$$e^{-2t} u_2 = -\frac{1}{3} e^{-3t} + C_1$$

$$u_2 = -\frac{1}{3} e^{-t} + C_1 e^{2t}$$

Sustitución de u_2 en ec.1:

$$u_1' = 2u_1 + \left(-\frac{1}{3} e^{-t} + C_1 e^{2t}\right) + t$$

$$u_1' - 2u_1 = -\frac{1}{3} e^{-t} + C_1 e^{2t} + t$$

$$P(t) = -2$$

$$u(t) = e^{\int -2 dt} = e^{-2t}$$

$$e^{-2t} u_1 = \int \left(-\frac{1}{3} e^{-3t} + C_1 + t e^{-2t}\right) dt$$

$$\begin{array}{c|c} \text{D} & \text{I} \\ \hline + & t \\ - & 1 \\ + & 0 \end{array} \quad \begin{array}{c} e^{-2t} \\ -e^{-2t}/2 \\ e^{-2t}/4 \end{array} \quad \begin{array}{c} \uparrow \\ \uparrow \\ \uparrow \end{array} = -\frac{1}{2} t e^{-2t} - \frac{1}{4} e^{-2t}$$

$$e^{-2t} u_1 = \frac{1}{9} e^{-3t} + C_1 t - \frac{1}{2} t e^{-2t} - \frac{1}{4} e^{-2t} + C_2$$

$$u_1 = \frac{1}{9} e^{-t} + C_1 t e^{2t} - \frac{1}{2} t - \frac{1}{4} + C_2 e^{2t}$$

Solución general:

$$u(t) = \begin{pmatrix} \frac{1}{9} e^{-t} + C_1 t e^{2t} - \frac{1}{2} t - \frac{1}{4} + C_2 e^{2t} \\ -\frac{1}{3} e^{-t} + C_1 e^{2t} \end{pmatrix}$$

Apéndice B

Bitácora de IA

Evidencia del uso de IA como copiloto en el desarrollo del proyecto.

Bitácora Uso de la IA

Prompt	Respuesta de la IA (resumen)
<i>“Dime exactamente qué es lo que debo hacer para la primera parte del proyecto”</i>	Se generó un resumen estructurado de las etapas 1.a a 1.d del proyecto, enfocándose en la implementación y validación de RK2/RK4.
<i>“Arregla f_segundo_orden ya que la cambié por la función $y'' + 4y = x^2$”</i>	Se corrigió la conversión del segundo orden a sistema, se generó solución analítica correcta y se ajustaron C_1 y C_2 para que coincidieran con condiciones iniciales.
<i>“Dame la solución del sistema 2x2 de forma vectorial/matricial”</i>	Se derivó la solución analítica del sistema ($u' = Au + f(t)$) usando el método general con matriz exponencial y se devolvió en forma vectorial y matricial.
<i>“Esta solución analítica está correcta?” (función y sistema lineal)</i>	Se revisaron las expresiones, se corrigieron errores de signos y términos, se explicó el origen de C_1 y C_2 , y se confirmó la versión correcta.
<i>“Modifica el código para que regrese la solución numérica de cada ecuación”</i>	Se reescribieron las funciones primer_orden(), segundo_orden(), sistema_lineal(), no_lineal() para que devolvieran datos numéricos útiles para gráficas y análisis.
<i>“¿Por qué el error del sistema lineal es tan grande?”</i>	Se detectó que C_1 y C_2 eran incorrectos; se corrigieron para que coincidieran con las condiciones iniciales y el error bajó de ~ 130 a $2e-5$.
<i>“Ayúdame con el estudio de convergencia”</i>	Se creó una función general convergencia(...), se explicó cómo evaluar soluciones exactas en los puntos del mallado, y se corrigieron errores de dimensiones.
<i>“Error: ValueError operands could not be broadcast...”</i>	Se identificó que la solución analítica del sistema lineal debía regresar forma (N,2); se implementó exacta_sistema_lineal() con np.column_stack.
<i>“Crea una sección del código para el estudio de convergencia”</i>	Se agregó el bloque completo: impresión de tablas, cálculo de errores RK2/RK4, y se generaron funciones para graficar convergencia log-log.
<i>“Ayúdame a crear las gráficas para cada función”</i>	Se generaron las funciones para graficar la solución numérica (RK2 Y RK4 y la solución analítica para cada una de las ecuaciones.
<i>“Traceback error por Mathtext en graficar_sistemalineal”</i>	Se identificó la ParseFatalException causada por el uso de entornos de matriz de LaTeX ($\begin{pmatrix}$ o $\begin{bmatrix}$) no soportados por Matplotlib. Se simplificó la notación de la matriz en el título a una forma lineal ($A = \begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix}$).
<i>“Verifica si mi código cumple con los requerimientos del proyecto final”</i>	Se verificó todo el proyecto contra el PDF del anteproyecto: RK2/RK4, 4 ecuaciones, solución analítica, comparación, gráficas, estudio de convergencia, sistema no lineal. Código marcado como completo y excelente.

Apéndice C

Código Fuente (Python)

A continuación se presenta el código principal utilizado ('proyecto_final.py').

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  # =====
5  # METODOS NUMERICOS
6  # =====
7  def rk2_heun(f, t0, y0, h, n_steps):
8      t = t0
9      y = np.array(y0, dtype=float)
10     ts = [t]
11     ys = [y.copy()]
12     for i in range(n_steps):
13         k1 = f(t, y)
14         y_euler = y + h * k1
15         k2 = f(t + h, y_euler)
16         y = y + (h/2.0)*(k1 + k2)
17         t += h
18         ts.append(t)
19         ys.append(y.copy())
20     return np.array(ts), np.array(ys)
21
22     def rk4(f, t0, y0, h, n_steps):
23         t = t0
24         y = np.array(y0, dtype=float)
25         ts = [t]
26         ys = [y.copy()]
27         for i in range(n_steps):
28             k1 = f(t, y)
29             k2 = f(t + h/2.0, y + h*k1/2.0)
30             k3 = f(t + h/2.0, y + h*k2/2.0)
31             k4 = f(t + h, y + h*k3)
32             y = y + (h/6.0)*(k1 + 2*k2 + 2*k3 + k4)
33             t += h
34             ts.append(t)
35             ys.append(y.copy())
36         return np.array(ts), np.array(ys)
37
38     # (El resto del código ha sido omitido por brevedad en el documento
39     # pero esta disponible completo en el repositorio GitHub).
```


Listing C.1: Implementación de Métodos Numéricos